

Screenshot:-1.1.4

CODETANTRA

Home

chitransh.phulkaybat025@snappru.edu.inSupportLogout

1.1.4. Minimum Spanning Tree using Kruskal's Algorithm02/23

Write a C program to construct the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Kruskal's Algorithm.

You are provided with a driver code. Complete the function **kruskalMST()** so that the program correctly finds and prints the Minimum Spanning Tree and its total cost. The graph is represented using an adjacency matrix.

Implement the function:

```
void kruskalMST(int **cost, int V)
```

Where:

- $V \rightarrow$ Number of vertices in the graph
- $cost \rightarrow$ Adjacency matrix storing edge weights

Input Format:

- The first line of input reads an integer V by prompting "No of vertices: ", representing the number of vertices in the graph.
- The next $V \times V$ integers represent the adjacency matrix of the graph with a heading "Adjacency matrix:"
- $cost[i][j]$ indicates the weight of the edge between vertex i and vertex j
- If there is no edge between two vertices, the weight should be given as 9999 (representing infinity).
- The graph is undirected, so the matrix is symmetric

Output Format:

1. The output should display each edge selected for the Minimum Spanning Tree in the order they are chosen in the below format.

```
Edge k: (u, v) cost: w
```

Where:

- k is the edge number starting from 0
- u and v are the vertices connected by the edge
- w is the weight of the edge

2. In the last line the program should print the total minimum cost of the MST as follows:

```
Minimum cost= X
```

Where X is the total cost of the Minimum Spanning Tree.

C minCost...

```
31 // If no cycle, include the edge
32 if (pu != pv) {
33     parent[pv] = pu;
34     printf("Edge %d: (%d, %d) cost: %d\n", edgeCount, u, v, min);
35     minCost += min;
36     edgeCount++;
37     // Remove the edge so it is not considered again
38     cost[u][v] = cost[v][u] = INT_MAX;
39 }
40 printf("Minimum cost= %d\n", minCost);
41
42 }
43
44 int main() {
45     int V;
46     printf("No of vertices: ");
47     scanf("%d", &V);
48
49     int **cost = (int **)malloc(V * sizeof(int *));
50     for (int i = 0; i < V; i++)
51         cost[i] = (int *)malloc(V * sizeof(int));
```

Average time0.119 sMaximum time0.132 s193.000 ms132.000 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1122 msDebug

Expected output

Actual output

No. of vertices: 5No. of vertices: 5

Adjacency matrix:Adjacency matrix:

9999 2 9999 9999 59999 2 9999 9999 52 9999 3 9999 99999999 3 9999 4 99999999 3 9999 4 99999999 9999 4 9999 9999

TerminalTest cases

< Prev

Reset

Submit

Next >

